File Edit Selection View Go Run Terminal Help

sy12

Update

EXPLORER

SY12

- .vscode
  - c_cpp_properties.json
  - launch.json
  - settings.json
- project
  - main.py
- assignment1.py (1).txt
- assignment2.py.txt
- assignment3.py
- IdeaSpark Poster Template.pptx
- paract assignment3.py
- pract assignment3.py
- pract1.py
- pract2.py
- pract3.py
- queue.c
- queue.exe
- report_generator1.py
- stackk.cpp
- stackk.exe
- unit2assignment-1.py

OUTLINE

TIMELINE

ract2.py | pract3.py | pract1.py | pract assignment3.py | assignment3.py | unit2assignment-1.py | fibonacci.py | sss.py

unit2assignment-1.py > ...

```
109     print(f"[4] Fast doubling     : {fib_fast_doubling(n)}")
110
111     print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
112     print([fib_dp(i) for i in range(15)])
113
114     # ---- timing comparison: naive recursion vs. iterative DP ---------
115     n_big = 28
116     t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
117     t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
118     print(f"\nTiming fib({n_big}):")
119     print(f"  naive recursion : {t_naive:.5f} sec")
120     print(f"  iterative DP    : {t_dp:.8f} sec  (thousands of times faster)")
121
122     # ---- a very large n: only practical with the efficient approaches -
123     n_huge = 100
124     print(f"\nFibonacci({n_huge}) via iterative DP  : {fib_dp(n_huge)}")
125     print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powersh...

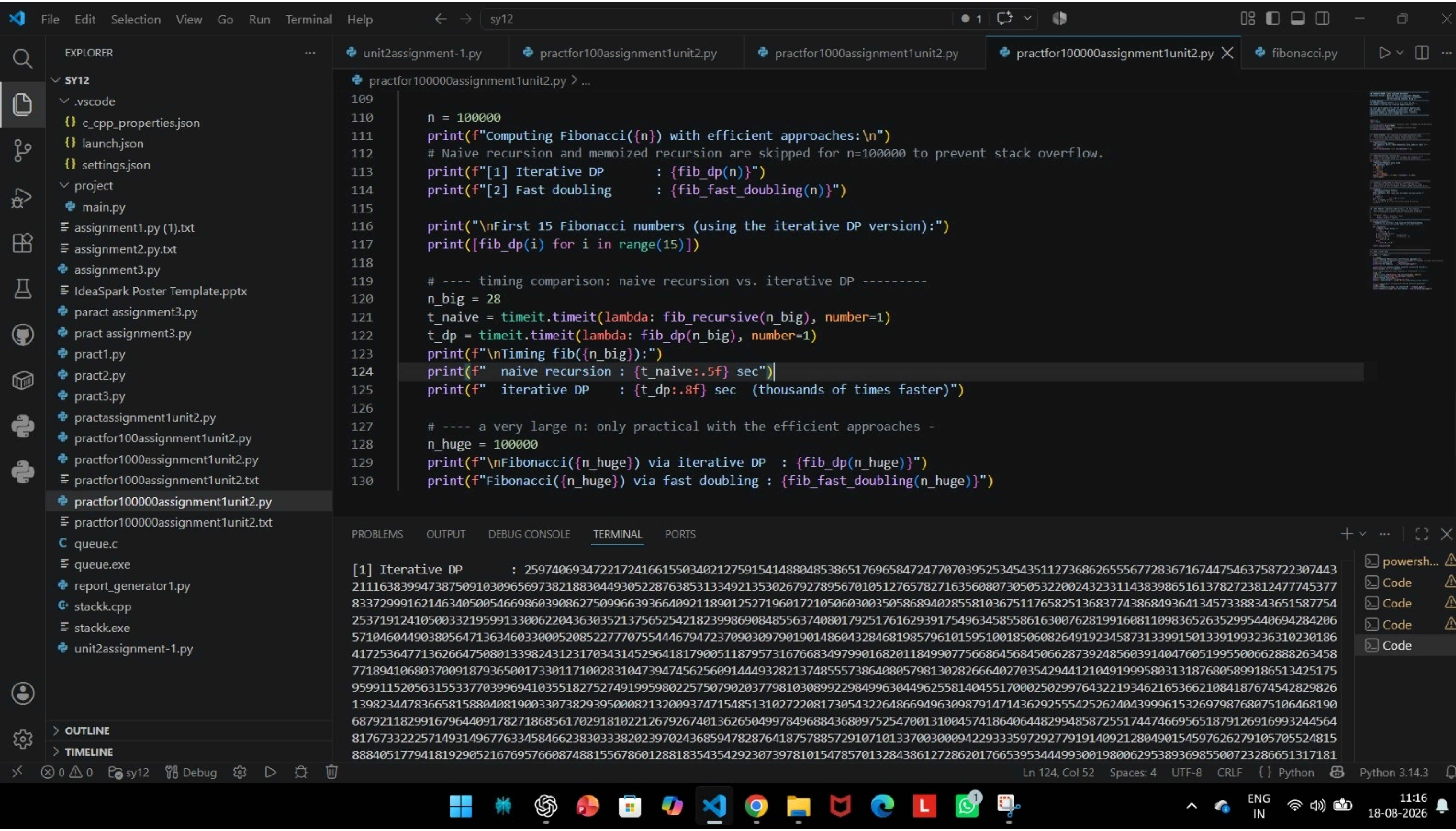
Code

```
Computing Fibonacci(10) with four different approaches:

[1] Naive recursion   : 55
[2] Memoized recursion : 55
[3] Iterative DP      : 55
[4] Fast doubling     : 55

First 15 Fibonacci numbers (using the iterative DP version):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]

Timing fib(28):
  naive recursion : 0.04591 sec
  iterative DP    : 0.00000400 sec  (thousands of times faster)

Fibonacci(100) via iterative DP  : 354224848179261915075
Fibonacci(100) via fast doubling : 354224848179261915075
PS C:\Users\Mo.Sajid\OneDrive\Desktop\sy12>
```

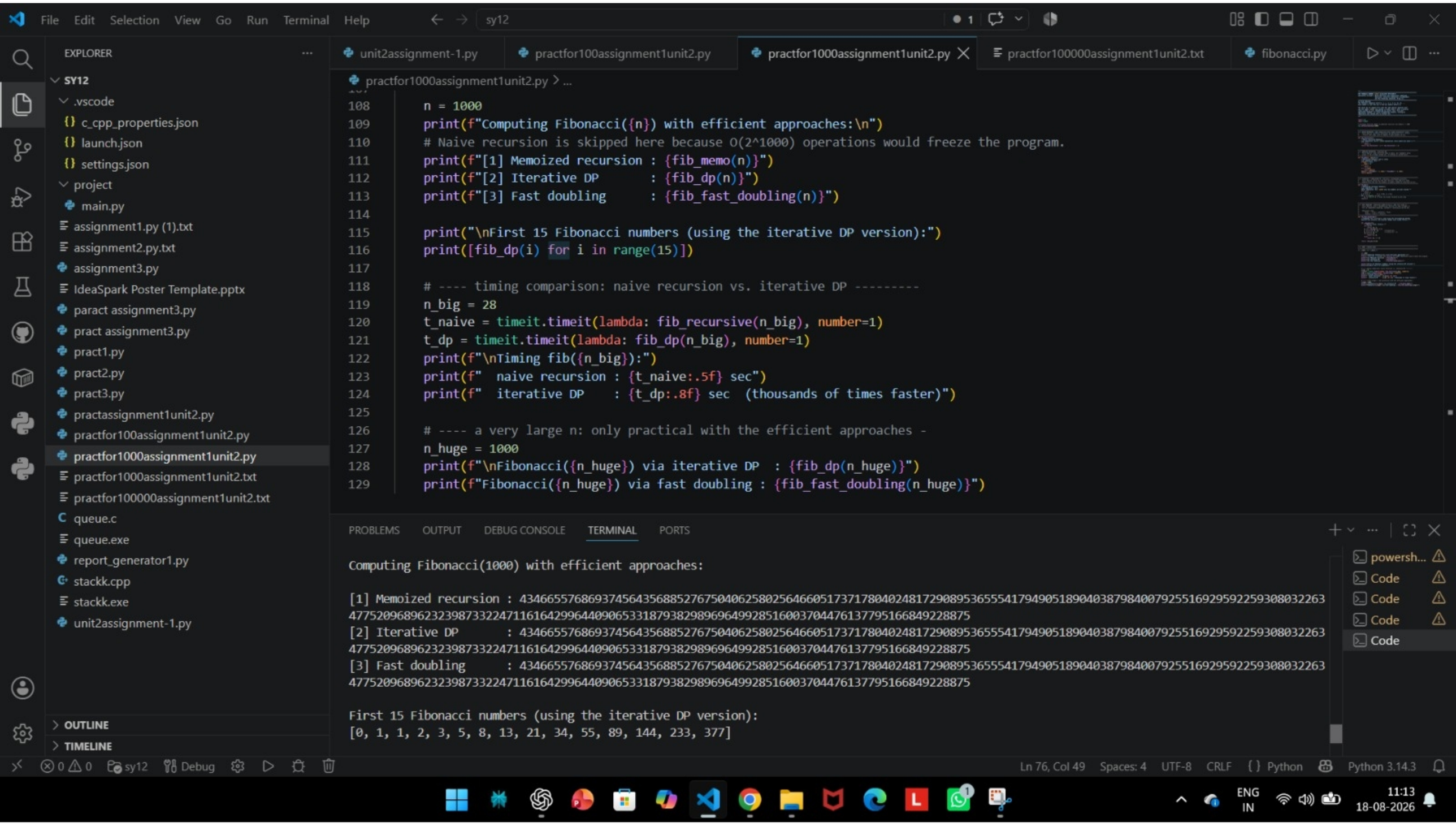
0 0 sy12 Debug

Ln 116, Col 68 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3

ENG IN 10:18 18-08-2026



File Edit Selection View Go Run Terminal Help
sy12
unit2assignment-1.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor100000assignment1unit2.py
fibonacci.py
EXPLORER
SY12
.vscode
c_cpp_properties.json
launch.json
settings.json
project
main.py
assignment1.py (1).txt
assignment2.py.txt
assignment3.py
IdeaSpark Poster Template.pptx
paract assignment3.py
pract assignment3.py
pract1.py
pract2.py
pract3.py
practassignment1unit2.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor1000assignment1unit2.txt
practfor100000assignment1unit2.py
practfor100000assignment1unit2.txt
queue.c
queue.exe
report_generator1.py
stackk.cpp
stackk.exe
unit2assignment-1.py
OUTLINE
TIMELINE
practfor100000assignment1unit2.py > ...
109
110 n = 100000
111 print(f"Computing Fibonacci({n}) with efficient approaches:\n")
112 # Naive recursion and memoized recursion are skipped for n=100000 to prevent stack overflow.
113 print(f"[1] Iterative DP : {fib_dp(n)}")
114 print(f"[2] Fast doubling : {fib_fast_doubling(n)}")
115
116 print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
117 print([fib_dp(i) for i in range(15)])
118
119 # ---- timing comparison: naive recursion vs. iterative DP ---------
120 n_big = 28
121 t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
122 t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
123 print(f"\nTiming fib({n_big}):")
124 print(f" naive recursion : {t_naive:.5f} sec")
125 print(f" iterative DP : {t_dp:.8f} sec (thousands of times faster)")
126
127 # ---- a very large n: only practical with the efficient approaches -
128 n_huge = 100000
129 print(f"\nFibonacci({n_huge}) via iterative DP : {fib_dp(n_huge)}")
130 print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[1] Iterative DP : 25974069347221724166155034021275915414880485386517696584724770703952534543511273686265556772836716744754637587223 07443
2111638399473875091030965697382188304493052287638531334921353026792789567010512765782716356080730505322002432331143839865161378272381247774537 7
833729991621463405005466986039086275099663936640921189012527196017210506030035058689402855810367511765825136837743868493641345733883436515877 54
253719124105003321959913300622043630352137565254218239986908485563740801792517616293917549634585586163007628199160811098365263529954406942842 06
571046044903805647136346033000520852277707554446794723709030979019014860432846819857961015951001850608264919234587313399150133919932363102301 86
417253647713626647508013398243123170343145296418179005118795731676683497990168201184990775668645684506628739248560391404760519955006628882634 58
771894106803700918793650017330117100283104739474562560914449328213748555738640805798130282666402703542944121049199958031318768058991865134251 75
959911520563155337703996941035518275274919959802257507902037798103089922984996304496255814045517000250299764322193462165366210841876745428298 26
139823447836658158804081900330738293950008213200937471548513102722081730543226486694963098791471436292555425262404399961532697987680751064681 90
687921182991679644091782718685617029181022126792674013626504997849688436809752547001310045741864064482994858725517447466956518791269169932445 64
817673322257149314967763345846623830333820239702436859478287641875788572910710133700300094229333597292779191409212804901545976262791057055248 15
888405177941819290521676957660874881556786012881835435429230739781015478570132843861272862017665395344499300198006295389369855007232866513171 81
powersh...
Code
Code
Code
Code
sy12 Debug
Ln 124, Col 52 Spaces: 4 UTF-8 CRLF {} Python Python 3.14.3
ENG IN
11:16
18-08-2026

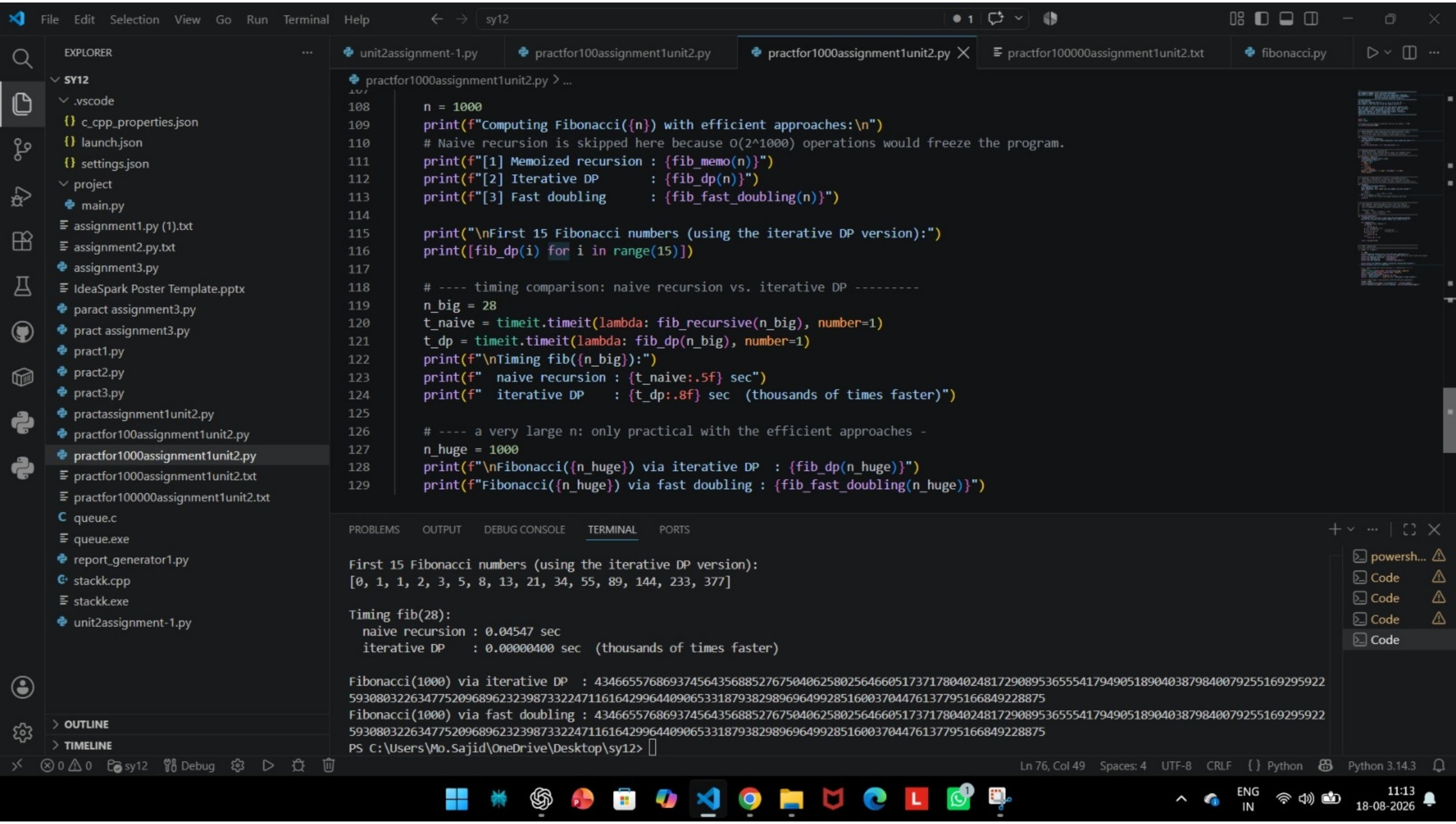

File Edit Selection View Go Run Terminal Help
sy12
EXPLORER
SY12
.vscode
c_cpp_properties.json
launch.json
settings.json
project
main.py
assignment1.py (1).txt
assignment2.py.txt
assignment3.py
IdeaSpark Poster Template.pptx
paract assignment3.py
pract assignment3.py
pract1.py
pract2.py
pract3.py
practassignment1unit2.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor1000assignment1unit2.txt
practfor100000assignment1unit2.txt
queue.c
queue.exe
report_generator1.py
stackk.cpp
stackk.exe
unit2assignment-1.py
OUTLINE
TIMELINE
unit2assignment-1.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor100000assignment1unit2.txt
fibonacci.py
practfor1000assignment1unit2.py > ...
n = 1000
print(f"Computing Fibonacci({n}) with efficient approaches:\n")
# Naive recursion is skipped here because O(2^1000) operations would freeze the program.
print(f"[1] Memoized recursion : {fib_memo(n)}")
print(f"[2] Iterative DP : {fib_dp(n)}")
print(f"[3] Fast doubling : {fib_fast_doubling(n)}")
print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
print([fib_dp(i) for i in range(15)])
# ---- timing comparison: naive recursion vs. iterative DP ---------
n_big = 28
t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
print(f"\nTiming fib({n_big}):")
print(f" naive recursion : {t_naive:.5f} sec")
print(f" iterative DP : {t_dp:.8f} sec (thousands of times faster)")
# ---- a very large n: only practical with the efficient approaches -
n_huge = 1000
print(f"\nFibonacci({n_huge}) via iterative DP : {fib_dp(n_huge)}")
print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Computing Fibonacci(1000) with efficient approaches:
[1] Memoized recursion : 43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922593080322634775209689623239873322471161642996440906533187938298969649928516003704476137795166849228875
[2] Iterative DP : 43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922593080322634775209689623239873322471161642996440906533187938298969649928516003704476137795166849228875
[3] Fast doubling : 43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922593080322634775209689623239873322471161642996440906533187938298969649928516003704476137795166849228875
First 15 Fibonacci numbers (using the iterative DP version):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
powersh...
Code
Code
Code
Code
0 0 sy12 Debug
Ln 76, Col 49 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3
ENG
IN
11:13
18-08-2026

practfor100000assignment1unit2.py

```python
    n = 100000
    print(f"Computing Fibonacci({n}) with eff(variable) n: Literal[100000]
    # Naive recursion and memoized recursion                               t stack overflow.
    print(f"[1] Iterative DP       : {fib_dp(n)}")
    print(f"[2] Fast doubling      : {fib_fast_doubling(n)}")

    print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
    print([fib_dp(i) for i in range(15)])

    # ---- timing comparison: naive recursion vs. iterative DP ---------
    n_big = 28
    t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
    t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
    print(f"\nTiming fib({n_big}):")
    print(f"  naive recursion : {t_naive:.5f} sec")
    print(f"  iterative DP    : {t_dp:.8f} sec  (thousands of times faster)")

    # ---- a very large n: only practical with the efficient approaches -
    n_huge = 100000
    print(f"\nFibonacci({n_huge}) via iterative DP  : {fib_dp(n_huge)}")
    print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
```

TERMINAL

```
[2] Fast doubling      : 2597406934722172416615503402127591541488048538651769658472477070395253454351127368626555677283671674475463758722307443
211163839947387509103096569738218830449305228763853133492135302679278956701051276578271635608073050532200243233114383986516137827238124777453777
833729991621463405005466986039086275099663936640921189012527196017210506030035058689402855810367511765825136837743868493641345733883436515877544
253719124105003321959913300622043630352137565254218239986908485563740801792517616293917549634585586163007628199160811098365263529954406942842066
571046044903805647136346033000520852277707554446794723709030979019014860432846819857961015951001850608264919234587313399150133919932363102301866
417253647713626647508013398243123170343145296418179005118795731676683497990168201184990775668645684506628739248560391404760519955006628882634588
771894106803700918793650017330117100283104739474562560914449328213748555738640805798130282666402703542944121049199958031318768058991865134251755
959911520563155337703996941035518275274919959802257507902037798103089922984996304496255814045517000250299764322193462165366210841876745428298266
139823447836658158804081900330738293950008213200937471548513102722081730543226486694963098791471436292555425262404399961532697987680751064681900
687921182991679644091782718685617029181022126792674013626504997849688436809752547001310045741864064482994858725517447466956518791269169932445644
817673322257149314967763345846623830333820239702436859478287641875788572910710133700300094229333597292779191409212804901545976262791057055248155
888405177941819290521676957660874881556786012881835435429230739781015478570132843861272862017665395344499300198006295389369855007232866513171811
```


File Edit Selection View Go Run Terminal Help
sy12
EXPLORER
SY12
.vscode
c_cpp_properties.json
launch.json
settings.json
project
main.py
assignment1.py (1).txt
assignment2.py.txt
assignment3.py
IdeaSpark Poster Template.pptx
paract assignment3.py
pract assignment3.py
pract1.py
pract2.py
pract3.py
practassignment1unit2.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor1000assignment1unit2.txt
practfor100000assignment1unit2.txt
queue.c
queue.exe
report_generator1.py
stackk.cpp
stackk.exe
unit2assignment-1.py
OUTLINE
TIMELINE
unit2assignment-1.py
practfor100assignment1unit2.py
practfor1000assignment1unit2.py
practfor100000assignment1unit2.txt
fibonacci.py
practfor1000assignment1unit2.py > ...
n = 1000
print(f"Computing Fibonacci({n}) with efficient approaches:\n")
# Naive recursion is skipped here because O(2^1000) operations would freeze the program.
print(f"[1] Memoized recursion : {fib_memo(n)}")
print(f"[2] Iterative DP : {fib_dp(n)}")
print(f"[3] Fast doubling : {fib_fast_doubling(n)}")
print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
print([fib_dp(i) for i in range(15)])
# ---- timing comparison: naive recursion vs. iterative DP ---------
n_big = 28
t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
print(f"\nTiming fib({n_big}):")
print(f" naive recursion : {t_naive:.5f} sec")
print(f" iterative DP : {t_dp:.8f} sec (thousands of times faster)")
# ---- a very large n: only practical with the efficient approaches -
n_huge = 1000
print(f"\nFibonacci({n_huge}) via iterative DP : {fib_dp(n_huge)}")
print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
First 15 Fibonacci numbers (using the iterative DP version):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]
Timing fib(28):
naive recursion : 0.04547 sec
iterative DP : 0.00000400 sec (thousands of times faster)
Fibonacci(1000) via iterative DP : 43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922
59308032263477520968962323987332247116164299644090653318793829896964992851600370447613779516684922887 5
Fibonacci(1000) via fast doubling : 43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922
59308032263477520968962323987332247116164299644090653318793829896964992851600370447613779516684922887 5
PS C:\Users\Mo.Sajid\OneDrive\Desktop\sy12>
powersh...
Code
Code
Code
Code
0 0 sy12 Debug
Ln 76, Col 49 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3
ENG IN
11:13 18-08-2026

File Edit Selection View Go Run Terminal Help

sy12

EXPLORER

- SY12
  - .vscode
    - c_cpp_properties.json
    - launch.json
    - settings.json
  - project
    - main.py
  - assignment1.py (1).txt
  - assignment2.py.txt
  - assignment3.py
  - IdeaSpark Poster Template.pptx
  - paract assignment3.py
  - pract assignment3.py
  - pract1.py
  - pract2.py
  - pract3.py
  - practassignment1unit2.py
  - practfor100assignment1unit2.py
  - practfor1000assignment1unit2.txt
  - practfor100000assignment1unit2.txt
  - queue.c
  - queue.exe
  - report_generator1.py
  - stackk.cpp
  - stackk.exe
  - unit2assignment-1.py

OUTLINE

TIMELINE

unit2assignment-1.py | practfor100assignment1unit2.py | practfor1000assignment1unit2.txt | practfor100000assignment1unit2.txt | fibonacci.py

practfor100assignment1unit2.py > ...

```python
    n = 100
    print(f"Computing Fibonacci({n}) with efficient approaches:\n")
    # Naive recursion is skipped here because O(2^100) operations would freeze the program.
    print(f"[1] Memoized recursion : {fib_memo(n)}")
    print(f"[2] Iterative DP       : {fib_dp(n)}")
    print(f"[3] Fast doubling      : {fib_fast_doubling(n)}")

    print("\nFirst 15 Fibonacci numbers (using the iterative DP version):")
    print([fib_dp(i) for i in range(15)])

    # ---- timing comparison: naive recursion vs. iterative DP ---------
    n_big = 28
    t_naive = timeit.timeit(lambda: fib_recursive(n_big), number=1)
    t_dp = timeit.timeit(lambda: fib_dp(n_big), number=1)
    print(f"\nTiming fib({n_big}):")
    print(f"  naive recursion : {t_naive:.5f} sec")
    print(f"  iterative DP    : {t_dp:.8f} sec  (thousands of times faster)")

    # ---- a very large n: only practical with the efficient approaches -
    n_huge = 100
    print(f"\nFibonacci({n_huge}) via iterative DP  : {fib_dp(n_huge)}")
    print(f"Fibonacci({n_huge}) via fast doubling : {fib_fast_doubling(n_huge)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Mo.Sajid\OneDrive\Desktop\sy12> python -u "c:\Users\Mo.Sajid\OneDrive\Desktop\sy12\practassignment1unit2.py"
Computing Fibonacci(100):

[1] Iterative DP   : 354224848179261915075
[2] Fast Doubling  : 354224848179261915075
[3] Memoized DP    : 354224848179261915075

Timing for 10,000 runs:
  Iterative DP  : 0.029727 sec
  Fast Doubling : 0.017216 sec
PS C:\Users\Mo.Sajid\OneDrive\Desktop\sy12>
```

powersh... Code Code Code Code

0 0 sy12 Debug Ln 124, Col 40 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3

ENG IN 11:09 18-08-2026